Akka vs. Databricks

Mosaic AI / Agent Bricks — a comparison for teams building agentic AI

October 2026

Databricks is a lakehouse with an agent layer attached — the data, feature, and model tier, **not a governed agentic runtime**. Agent Bricks builds agents that read your Lakehouse and run on Agent Runtime, a Databricks Apps service that recovers interrupted runs and keeps their state in Lakebase Postgres. Unity Gateway policies (Beta) can block a tool call or hold it for approval, and Databricks leaves EU AI Act risk assessment to the customer. The Akka Agentic AI Platform is the governed agentic runtime that guarantees resilience and scalability — durable in-memory state, a six-nines SLA, inline enforcement — and complements the lakehouse beneath it.

99.9%	99.9999%	4ms	1,718
DATABRICKS CONTROL-PLANE SLA	AKKA AGENTIC AI PLATFORM SLA	AKKA STATE READS	CONTROLS ENFORCED PRE-DEPLOY

DIMENSION	DATABRICKS (MOSAIC AI / AGENT BRICKS)	AKKA
What it is	A data intelligence platform (lakehouse) with an agent layer — Agent Bricks, Agent Runtime, Model Serving, AI Search (formerly Vector Search), Genie	The Akka Agentic AI Platform, a platform that guarantees resilience and scalability — agents, orchestration, memory, streaming, APIs, and governance on one runtime
Primary role	The data, feature, and model tier agents read from	The runtime agents run on
Agent state	Agent Runtime on Databricks Apps recovers interrupted runs; run state, sessions, and long-term memory in Lakebase (managed Postgres), session re-read each turn	Durable sharded in-memory — 4ms reads / sub-10ms writes, replayable from the event journal
Availability SLA	99.9% control plane; Model Serving rides the Databricks SLA, no published six-nines agent-tier number	99.9999% across the entire platform, contractual, backed by indemnities
HA / DR	Customer-architected, or gated managed DR that fails over in minutes; most teams run active-passive; example targets RTO < 1h, RPO < 15min (critical); Model Serving endpoints not in managed DR	Active-active; sub-1-minute RTO; zero-byte RPO — Akka owns it
Governance / EU AI Act	Unity Catalog lineage, access control, audit; Unity Gateway policies (Beta) block a tool call or require approval; DAGF/DASF guidance; EU AI Act assessment left to the customer	Runtime enforcement: inline guardrails, hash-chained evidence, pre-deploy classification, sealed posture, Akka Verify
Coupling	Agents bound to the Lakehouse, Unity Catalog, and DBU-metered serving	Deploy anywhere — Akka cloud, your VPC, your Kubernetes, on-prem, sovereign cloud; portable specs
Cost model	DBU consumption metering (per-token + provisioned-throughput DBUs); scales with load	Shared compute on one runtime; up to 90% lower infrastructure for the same workload; fixed annual fee
Maturity	Agent Bricks beta Jun 2025; Supervisor Agent GA Feb 2026, legacy with Knowledge Assistant since Oct 12 2026; agent stack rebuilt around Agent Runtime in Oct 2026	since 2009, 100,000+ production deployments, 52 banks

A Data Platform With Agents, Not a Governed Agentic Runtime

Databricks is a lakehouse — Delta Lake storage, Unity Catalog governance, and Mosaic AI on top — with an agent layer attached. Agent Bricks, AI Search (formerly Vector Search), and Genie build agents that read your governed data; Agent Runtime on Databricks Apps runs them and recovers interrupted runs. That is the data, feature, and model tier. Databricks publishes no six-nines availability figure for the tier agents run on. The line is structural: a lakehouse optimizes storage, retrieval, lineage, and model access; an agentic runtime optimizes durable execution, in-memory state, failover, and inline policy enforcement on running processes. Databricks delivers the first and attaches agents to it; Akka is the second.

Capability	Databricks	Akka
Lakehouse / feature store / model serving	Yes — core strength	Not an Akka layer (complement)
Vector search / semantic retrieval	Yes — Databricks AI Search (formerly Vector Search)	Not an Akka layer (complement)
Native durable agents with in-memory state	Run recovery on Agent Runtime; state in Lakebase Postgres, not in memory	Built in — durable sharded in-memory
Six-nines platform availability SLA	Not published for the agent tier	99.9999%, contractual
Active-active HA/DR, sub-1-min RTO, zero RPO	Customer-architected or gated managed DR; serving endpoints excluded from managed DR	Built in — Akka owns it
Inline runtime EU AI Act enforcement	Unity Gateway policies (Beta) block tool calls or require approval; EU AI Act assessment left to the customer	Built in — aspect-woven

Reliability: Agent State and the SLA That Covers It

Akka publishes a **99.9999%** availability SLA across the entire platform — contractual and backed by indemnities — with sub-1-minute RTO, zero-byte RPO, and active-active HA/DR that Akka owns and operates 24/7. Databricks publishes a 99.9% control-plane SLA; Model Serving is "backed by the Databricks SLA," but no six-nines number is published for the agent-serving tier. Disaster recovery is customer-architected — most teams run active-passive, with example targets of RTO under 1 hour and RPO under 15 minutes for critical workloads. Managed disaster recovery requires an access request, fails over in minutes, and does **not replicate Model Serving endpoints**.

The deeper difference is agent state. Databricks Agent Runtime recovers interrupted runs from run state kept in a Databricks-managed Lakebase database, and the agent reads its session from Lakebase managed Postgres at the start of every turn. Akka holds agent state in durable sharded in-memory storage — 4ms reads, sub-10ms writes, replayable from its event journal — so working memory survives failover without a per-turn database round trip.

Metric	Databricks	Akka
Availability SLA	99.9% control plane; no published agent-tier six-nines	99.9999% — entire platform
Allowed downtime / year	~8.7 hours (at 99.9%)	~31 seconds
RTO	< 1 hour (customer-architected, critical workloads)	Sub-1 minute
RPO	< 15 minutes (customer-architected)	Zero byte
HA/DR posture	Mostly active-passive; managed DR gated; serving endpoints not replicated	Active-active, Akka-owned
Agent state	Run state and sessions in Lakebase Postgres, session re-read each turn	Durable in-memory, 4ms / sub-10ms

Up to 90% Cheaper to Operate

AI systems built with Akka are up to **90% cheaper to operate** than Python-based systems — a function of the infrastructure required to run the same agentic transaction volume, not list price. The drivers are actor concurrency (~10T tokens/core/year vs ~2T; ~80% less compute than Python frameworks), shared compute, and micro-checkpointing. Manulife reported up to 300% more concurrency and 30–50% faster processing after porting from Python.

Databricks bills on **DBU consumption** — pay-per-token for Foundation Model APIs plus provisioned-throughput DBUs for hosted models and serving — so the meter moves with load, and the lakehouse, AI Search, and serving each consume separately. Akka runs all of it on one shared-compute runtime for a fixed annual fee finance can forecast.

Gateway Policies and Data Governance

Unity Catalog enforces access control, captures column-level lineage, logs activity for audit, and governs agents, tools, and models as catalog assets with on-behalf-of access controls. Unity Gateway, GA since August 4, 2026, adds service policies (Beta) that check an MCP tool call before it runs and allow it, block it, or require approval. Those policies cover calls routed through Unity Gateway. Databricks publishes the AI Governance Framework (5 pillars, 43 considerations) and the AI Security Framework, and leaves EU AI Act high-risk assessment to the customer.

The EU AI Act expects enforcement at the runtime: prohibited practices blocked as they occur, immutable records witnessed as decisions happen, human override of running processes, pre-deployment classification, and a retained evidence trail. Akka embeds all of this in the runtime.

Continuous AI Intelligence

Akka Optimize keeps improving the AI after it ships. It runs an evaluation loop on your own production traffic, grading live interactions and training smaller specialized models on your proprietary data, so the system keeps getting better as it runs. The loop delivers three results that compound over time:

- **Model leverage** — route each request to the best model from any vendor under your policies, and reserve frontier models for the requests that need them.
- **Data sovereignty** — the specialized models are trained on your proprietary data, owned by you, and run inside your own environment.
- **Cost governance** — full visibility and control of AI spend, with savings that compound as the loop keeps running.

The evaluations run continuously inside Akka and grade traffic from agents wherever they run, in Akka or in third-party harnesses.

Governance and the EU AI Act

Akka enforces governance in the runtime: inline guardrails, policies, LLMs-as-a-judge, and sanitizers; **hash-chained immutable evidence**; HITL/HOTL control; pre-deployment classification against 241 regulations and frameworks and 1,718 controls (699 carrying a financial penalty); multi-persona sign-offs; a sealed Governance Posture Package; and Akka Verify, which proves conformance from the running system. Unity Gateway governs the calls routed through it; Akka Verify applies any control to any AI system.

The penalties are enforceable now

Violation	Maximum Fine
Prohibited AI practices (Art. 5)	EUR 35M or 7% global turnover
High-risk obligations (Art. 9-15)	EUR 15M or 3% global turnover
Incorrect / misleading information	EUR 7.5M or 1.5% global turnover

High-risk AI carries a 10-year logging-retention obligation (Art. 72), enforceable since Feb 2025 / Aug 2025.

Gateway policies and the EU AI Act

Unity Catalog decides *who may read which data and where it came from*, and Unity Gateway policies can block or hold a tool call routed through the gateway. The EU AI Act also asks *whether the system was classified before it shipped and whether the evidence is sealed and immutable.* Databricks leaves that classification to the customer and keeps audit logs, usage tables, and inference tables.

Two Lifecycles, One Certified System

Building agents on Databricks is a developer-and-data-team workflow: define an agent in Agent Bricks, wire it to Lakehouse context and Unity Catalog permissions, deploy to Agent Runtime on Databricks Apps, evaluate with MLflow judges. Governance runs as a parallel track of Unity Catalog permissions and Unity Gateway policies. Akka runs two independent lifecycles on one platform via **Akka Specify**:

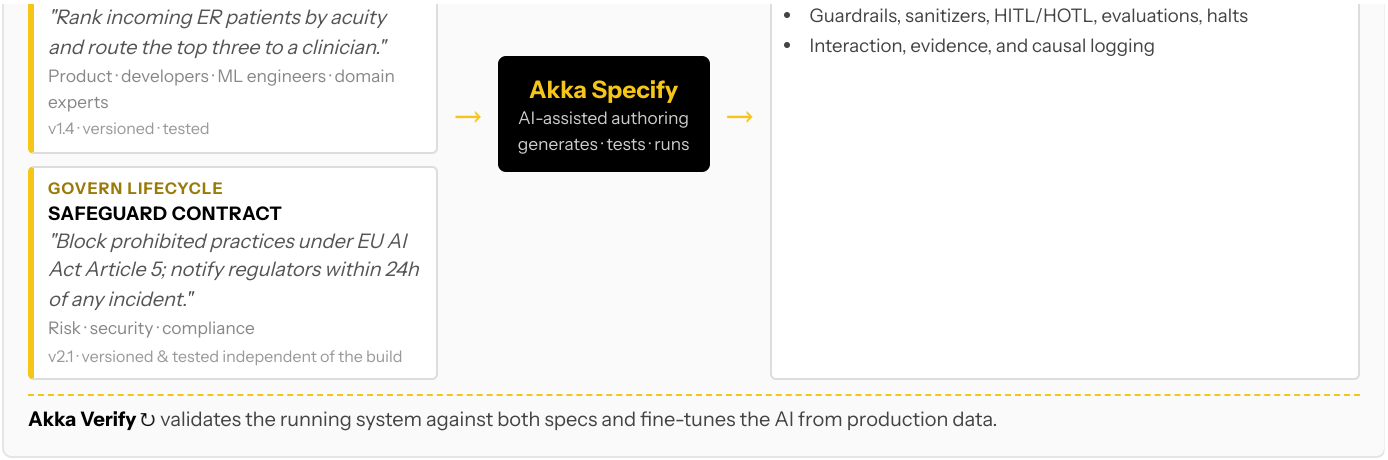
BUILD LIFECYCLE

FUNCTIONAL CONTRACT

ONE CERTIFIED AI SERVICE

Built, governed, and running

- Agents, tools, orchestration, memory, APIs, streaming, UI



The build and governance lifecycles are versioned and tested independently, by different audiences — a workflow Databricks runs as Unity Catalog permissions and Unity Gateway policies, without a safeguard specification versioned apart from the build.

Real-Time Streaming at Petabyte Scale

Akka's streaming is built into the runtime — continuous, backpressured, **petabyte-scale in-memory**, with no external broker — powering both agent feedback loops and high-throughput data processing (the engine behind Tubi's real-time hyper-personalization at 5 billion tokens per second). On Databricks, real-time data movement lives in the lakehouse (streaming tables, pipelines); feeding it into the agent layer is an integration the customer assembles across Agent Runtime, AI Search sync, and Lakebase, each metered and operated separately.

For the Buyer: Product Maturity, Coupling, and Accountability

The Databricks lakehouse is mature and widely adopted. The question for an agentic-AI decision is the **agent product's** maturity and what standardizing on it commits you to — not Databricks' viability, which is not in question.

Buyer concern	Databricks (Mosaic AI / Agent Bricks)	Akka
Agent product maturity	Agent Bricks announced beta Jun 2025; Supervisor Agent GA Feb 2026, then moved to legacy with Knowledge Assistant on Oct 12 2026; agent docs rebuilt around Agent Runtime and DurableAgentServer in Oct 2026	since 2009; 100,000+ production deployments; 52 banks
Coupling / lock-in	Agents bound to the Lakehouse, Unity Catalog, and DBU-metered serving; value compounds as more of the estate sits in Databricks	Deploy anywhere — Akka cloud, your VPC, your Kubernetes, on-prem, sovereign cloud; portable specs; BSL licensing
Who owns the SLA	Customer architects HA/DR or requests gated managed DR; serving endpoints excluded from managed DR	Akka owns the SLA, 24/7 SRE, one platform
Certifications	SOC 2 Type II, ISO 27001, ISO 27701, PCI DSS, HIPAA, HITRUST (Azure)	19 standards — SOC 2 II + public SOC 3, ISO 27001/42001, HIPAA, PCI DSS, GDPR, CCPA, PIPEDA, NIS2, DORA, EU AI Act, NIST AI RMF — plus annual pen tests, SBOMs, 40+ policies (trust.akka.io)
Risk transfer	Standard cloud terms	Availability and data-integrity guarantees backed by contractual indemnities
Budget predictability	DBU consumption metering that scales with load	Fixed annual fee finance can forecast

Akka complements your lakehouse

Akka is explicitly **not** a vector database, a semantic knowledge layer, a model-serving / inference engine, or a context graph — those are exactly what the Databricks lakehouse, Unity Catalog, AI Search, and Model Serving provide. For those layers Databricks sits *beneath* Akka, and the two are complementary: keep your governed data and feature/model tier in Databricks, and run the governed agentic system on Akka above it. The competitive overlap is narrow and specific — Agent Bricks, Agent Runtime, and Unity Gateway as the agent runtime and governance layer — and that is where the runtime-versus-data-platform line is drawn.

Customers Running Agentic and Real-Time Systems on Akka

Manulife	Tubi	Swiggy	John Deere	Verizon
2,000	**5B tok/s**	**71ms**	**1,000+**	**750%**
developers across 100 projects on one governed platform	real-time hyper-personalization engine	order-assignment AI, ~50% latency reduction	tractor sensors turned into real-time insight	order-processing capacity gain; 6s → 2.4s response

Common Questions

We already run Databricks. Why add Akka?
Keep it. Databricks is a strong lakehouse, and Akka complements it — Akka is not a vector DB, model-serving, or semantic layer. The gap is the agentic runtime: durable in-memory agent state, a six-nines platform SLA, active-active failover, and inline EU AI Act enforcement. Run your governed data and models on Databricks and the governed agentic system on Akka above it.

Agent Bricks is "governed." Isn't that the same governance?
Agent Bricks governs agents, tools, and models as Unity Catalog assets, and Unity Gateway policies (Beta) can block a tool call or hold it for approval before it runs. Those policies cover calls routed through Unity Gateway, and Databricks leaves EU AI Act high-risk assessment to the customer, with no pre-deployment classification. Akka Verify applies any control to any AI system and guarantees its enforcement, including pre-deployment classification against the regulations that apply.

Doesn't Databricks have agent memory now?
Databricks offers managed agent sessions and memory in Lakebase (managed Postgres), and Agent Runtime recovers interrupted runs; the agent reads its session from that database at the start of every turn. Akka holds agent state in durable sharded in-memory storage — 4ms reads, sub-10ms writes, replayable from its event journal — so working memory survives failover without a per-turn database round trip.

Is Databricks cheaper because we already pay for it?
Databricks bills on DBU consumption — pay-per-token plus provisioned-throughput DBUs for serving — so the agent meter scales with load and each layer consumes separately. Akka runs the whole agentic system on one shared-compute runtime, up to 90% cheaper to operate for the same workload, on a fixed annual fee.

Sources

Agent Bricks (what / GA): databricks.com/blog/introducing-agent-bricks (beta, Jun 2025); databricks.com/blog/agent-bricks-supervisor-agent-now-ga-orchestrate-enterprise-agents (Supervisor Agent GA, Feb 10 2026); developers.databricks.com/docs/agents/overview — agents/tools/models governed via Unity Catalog + on-behalf-of access
Model Serving (earlier agent deployment path): databricks.com/product/model-serving; docs.databricks.com/.../agent-framework/deploy-agent — agents deployed to serving endpoints; "backed by the Databricks SLA," serverless HA
Agent runtime / state / memory (checked 2026-10-26): docs.databricks.com/aws/en/agents/concepts/faq; docs.databricks.com/gcp/en/agents/deploy/agent-server; docs.databricks.com/aws/en/agents/custom-agents/stateful-agents — Agent Runtime on Databricks Apps; DurableAgentServer recovers interrupted runs; sessions and memory in Lakebase; docs.databricks.com/aws/en/release-notes/product/2026/october — Knowledge Assistant and Supervisor Agent legacy (Oct 12 2026)
Unity Gateway, AI Search, managed DR (checked 2026-10-26): databricks.com/blog/unity-ai-gateway-generally-available (GA Aug 4 2026); docs.databricks.com/aws/en/ai-gateway/govern-mcp-service — service policies (Beta) allow, block, or require approval on MCP tool calls; docs.databricks.com/aws/en/ai-search/ai-search — AI Search (formerly Vector Search); docs.databricks.com/aws/en/admin/managed-disaster-recovery — gated managed DR
Unity Catalog / Vector Search / Genie: databricks.com/product/unity-catalog — centralized access control, lineage, audit (data governance); learn.microsoft.com/.../vector-search/vector-search
SLA / DR: docs.databricks.com/.../reliability/best-practices — 99.9% control-plane SLA; docs.databricks.com/aws/en/admin/disaster-recovery & managed-disaster-recovery — typically active-passive, RTO < 1h / RPO < 15min (critical), "Model serving endpoints are not replicated in Databricks managed disaster recovery"
Pricing (DBU): databricks.com pricing; flexera.com/blog/finops/databricks-pricing-guide — DBU consumption; pay-per-token Foundation Model APIs + provisioned-throughput DBUs for serving
Governance / EU AI Act: databricks.com/trust/responsibleAI; databricks.com/blog/introducing-databricks-ai-governance-framework — DAGF (5 pillars, 43 considerations), DASF; committed to EU AI Act compliance
Certifications: databricks.com/legal/security-addendum; databricks.com/trust/compliance/soc — SOC 2 Type II, ISO 27001, ISO 27701, PCI DSS, HIPAA; HITRUST (Azure Databricks)
Akka facts (akka-facts.md, verified 2026-06-19): 99.9999% availability, active-active HA/DR, sub-1-min RTO, zero-byte RPO, contractual indemnities; durable in-memory 4ms / sub-10ms, replayable journal; ~10T vs ~2T tokens/core, ~80% less compute, up to 90% cheaper (akka.io/blog/go-slow-to-go-fast); Manulife up to 300% more concurrency, 30–50% faster; 241 regulations and frameworks / 1,718 controls / 699 with financial penalty; 19 standards (trust.akka.io); since 2009, 100,000+ deployments, 52 banks; profitable, Dell Technologies Capital; Tubi 5B tok/s, Swiggy 71ms, John Deere 1,000+ sensors, Verizon 750%